

SecureConvert — Everything You Must Know To Get A Good Grade

[Complete Guide — Everything You Must Know For A Good Grade](#)

Complete Defense & Grading Guide — For Prof. Armando Ruggeri

****Student: Kheizaran Nazari Khakeshoori — 2026-09-09 — Branch local_commit****

****How to use:**** Print this (Ctrl+P in docs/schema.html + this PDF) and keep beside laptop during presentation. Every bullet is a grading point.

1. Elevator Pitch (30 seconds — start with this)

"SecureConvert is an authenticated, RAM-only file converter. Users register, log in with password + TOTP, upload a file, we validate its binary signature (not extension), convert strictly in memory (md!'HTML/PDF, JSON!'CSV, PNG!'WebP), stream back, log to SQLite, and keep a full audit trail. No file ever touches disk. The focus is zero-trust handling + relational DB design."

****Portfolio line:**** Demonstrates *secure file handling + relational DB (FK, UNIQUE, indexes, migrations, transactions) + Express hardening + ephemeral processing.*

2. How To Run (Live Demo Script — Do This First)

****Prereq:**** Node 22, Python 3 (or just Node via `serve`), Fedora terminal.

1. env (first time)

[Complete Guide — Everything You Must Know For A Good Grade](#)

cp server/.env.example server/.env.local

edit server/.env.local -> JWT_SECRET (generate):

[Complete Guide — Everything You Must Know For A Good Grade](#)

node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"

2. API (terminal 1)

[Complete Guide — Everything You Must Know For A Good Grade](#)

cd server && npm install && npm start

-> Ø=Ý SecureConvert Server running on http://127.0.0.1:5000

[Complete Guide — Everything You Must Know For A Good Grade](#)

curl http://127.0.0.1:5000/api/health # must return {"status":"ok"}

3. Client (terminal 2, auto-opens browser)

[Complete Guide — Everything You Must Know For A Good Grade](#)

cd client && npm install && npm run dev

-> http://127.0.0.1:5500

[Complete Guide — Everything You Must Know For A Good Grade](#)

fallback: python3 -m http.server 5500

[Complete Guide — Everything You Must Know For A Good Grade](#)

4. Demo flow in browser

[Complete Guide — Everything You Must Know For A Good Grade](#)

Register -> save mfaSecret -> Step1 password -> Step2 TOTP
(Google Authenticator or totp.danhersam.com) -> Drag sample.md
-> choose pdf -> Upload & Convert -> Download -> History updates

[Complete Guide — Everything You Must Know For A Good Grade](#)

5. DB inspect (no sqlite3 CLI needed)

[Complete Guide — Everything You Must Know For A Good Grade](#)

npm run seed --prefix server # adds ada/lin/grace + 7 jobs

npm run db:inspect --prefix server # shows tables

xdg-open docs/schema.html # light printable ER

****If browser not auto-opening:**** `client/dev-serve.js` does `xdg-open`; if `serve -o` error seen before, you are on fixed version (shows light schema).

3. Architecture (Draw This On Board If Asked)

****Stack:**** Node 22 / Express 4 / SQLite3 / Sharp / PDFKit / Speakeasy TOTP / bcryptjs / JWT / Helmet / multer

****Diagram to quote:****

Browser (:5500) --CORS--> Express (:5000) [helmet + rateLimit]

-> /api/auth/register (bcrypt -> users + audit_logs)

-> /api/auth/login-step1 (tempToken -> users.step3Token)

-> /api/auth/login-step2 (speakeasy.verify -> JWT jti + sessions tokenHash=SHA256)

-> /api/convert (Bearer + multer.memoryStorage + validateMagicBytes -> converter -> audit+conversion_jobs -> blob)

-> /api/history (Bearer -> jobs)

-> /api/health

****Key file map:****

- `server/server.js:32` Express + health + 404 + error + graceful shutdown
 - `server/config.js:1` dotenv + JWT_SECRET guard + CORS_ORIGINS
 - `server/authRoutes.js:1` register/login transaction + audit + rateLimit
 - `server/authMiddleware.js:5` verify jti+userId
 - `server/userDb.js:10` SQLite WAL + migrations + FK
 - `server/converter.js` + `server/fileValidator.js` + `server/inputValidation.js`
 - `client/app.js:22` API_BASE, progress, `clearAuth` on 401, history
-

4. Database — This Is What The Professor Grades Most

4.1 ER — Memorize This

users 1 — " sessions (ON DELETE CASCADE)

users 1 — " conversion_jobs (ON DELETE CASCADE)

users 1 — " audit_logs (ON DELETE SET NULL — keep audit after delete)

Show `docs/schema.html` light version + `docs/db-schema.md:7` Mermaid.

4.2 Tables — Quote DDL From Memory

****users**** `(id PK AUTOINCREMENT, email UNIQUE NOT NULL, passwordHash, mfaSecret, mfaEnabled, step3Token)` + `idx\_users\_email`

- UNIQUE at DB: prevents race duplicate beyond JS. Index speeds `WHERE email=?`.

****sessions**** `(id TEXT PK=jti, userId FK, tokenHash UNIQUE, expiresAt, createdAt)` + `idx\_sessions\_user\_id, idx\_sessions\_expires\_at`

- `tokenHash=SHA256(JWT)` — leaked DB "" bearer token. `expiresAt` indexed for `WHERE expiresAt>now` and `DELETE where expiresAt<=now`.

****conversion_jobs**** `(id PK, userId FK, originalFilename, sourceType, targetType, status, createdAt)` + `idx\_conversion\_jobs\_user\_id, idx\_conversion\_jobs\_created\_at`

****audit_logs**** `(id PK, userId FK, action, ipAddress, createdAt)` + `idx\_audit\_logs\_user\_id, idx\_audit\_logs\_action`

- Keeps trail after delete.

****Pragmas**** `PRAGMA foreign\_keys=ON`, `journal\_mode=WAL` (readers+writer concurrent), `user\_version=2` (MIGRATIONS).

4.3 Migrations — Not Just IF NOT EXISTS

`server/userDb.js:10`:

```
const MIGRATIONS = [ `v1 base`, `v2 audit_logs + indexes` ];
```

```
// runMigrations() reads PRAGMA user_version, applies sequentially, PRAGMA user_version = 2
```

4.4 Five Queries — Be Ready To Type These

-- Q1 login

```
SELECT * FROM users WHERE email = ?; -- parameterized, uses idx_users_email
```

-- Q2 history GET /api/history

```
SELECT id,originalFilename,sourceType,targetType,createdAt
```

```
FROM conversion_jobs WHERE userId=? ORDER BY createdAt DESC LIMIT ?; -- safeLimit 1..100
```

-- Q3 cleanup

```
DELETE FROM sessions WHERE expiresAt <= ?;
```

-- Q4 audit

```
SELECT id,action,ipAddress,createdAt FROM audit_logs WHERE userId=? ORDER BY createdAt DESC LIMIT ?;
```

-- Q5 professor JOIN + GROUP BY

```
SELECT u.email, COUNT(j.id) AS conversions, MAX(j.createdAt) AS lastActive
```

```
FROM users u LEFT JOIN conversion_jobs j ON j.userId=u.id GROUP BY u.id ORDER BY conversions DESC;
```

```
SELECT targetType, COUNT(*) FROM conversion_jobs GROUP BY targetType;
```

Plus `EXPLAIN QUERY PLAN SELECT \* FROM conversion\_jobs WHERE userId=1 ORDER BY createdAt DESC LIMIT 20;` to show index impact.

4.5 Integrity Tricks To Mention

- `ALLOWED\_USER\_COLUMNS` whitelist in `updateUser` — blocks `email`; DROP `column` injection.
 - `?` everywhere — no string interpolation.
 - `transaction() { BEGIN IMMEDIATE; ... COMMIT/ROLLBACK }` makes `createUser + updateUser(mfaSecret)` atomic — ACID demo.
 - `LIMIT` sanitized — prevents `LIMIT 999999` DoS.
 - Seed via `server/seed.js:1` — `npm run seed` adds demo without `sqlite3` CLI (important on Fedora).
-

5. Authentication — Two Steps, Quote This

```
1. **Register:** `validateEmail + validatePassword(>=8)` `!` `check` `findByEmail` `!` `bcrypt.hash(12)` `!` `transaction(createUser + updateUser mfaSecret/`  
mfaEnabled=1)` `!` `audit register_success` `!` `return` `201 {mfaSecret, otpauthUrl}`.  
2. **Step1:** `findByEmail + bcrypt.compare` `!` `crypto.randomBytes(32)=tempToken` in `step3Token` `!` `audit login_step1_success`.  
3. **Step2:** `check` `step3Token==tempToken` + `speakeasy.totp.verify(window:1)` `!` `clear` `step3Token` `!` `jwt.sign({userId}, JWT_SECRET, {expiresIn:'1h',`  
jwtid:UUID})` `!` `sessions(tokenHash=SHA256)` `!` `audit login_success`.  
4. **Middleware:** `requireAuth` checks `Bearer` + `jwt.verify` + `findSession(jti, token)` + `userId==payload.userId`.  
**Google Authenticator offline:** after Register copy secret `!` add to app via "Enter setup key" or test online at `totp.danhersam.com` /  
`gauth.apps.gbraad.nl` (warn: test only). Or CLI: `node -e "console.log(require('speakeasy').totp({secret:'...',encoding:'base32'}))"`.
```

6. File Validation & Conversion — Security To Emphasize

- **Magic bytes** `fileValidator.js:11` — PNG `89 50 4E 47`, JPG `FF D8 FF`, PDF `%PDF`; rejects `0x00`, `ÿÿ`, empty after BOM. Before! just ext checks; now binary header + first 512B null scan + 1k text heuristic.
- **Filename** `inputValidation.js:8` — blocks `..`, `/`, `\`, control chars, checks alphanumeric.
- **Target matrix**: `html/pdf` needs `txt/json`, `csv` needs `json`, `webp` needs `png`; else 400 with clear msg.
- **Limits**: `MAX\_UPLOAD\_SIZE 10MB` (multer), JSON 2M chars /10k rows /100 headers, PNG 25M pixels `sharp({failOnError:true})`.
- **Ephemeral**: `multer.memoryStorage()` never writes; `conversion\_jobs` only after success + `audit convert\_txt\_to\_pdf`.

Demo injection block:

```
curl -X POST http://127.0.0.1:5000/api/auth/register -H "Content-Type: application/json" -d '{"email":"a@b.com" OR 1=1--","password":"12345678"}'
```

! 400 Provide valid email

[Complete Guide — Everything You Must Know For A Good Grade](#)

7. Testing & Verification — Numbers To Quote

- ``cd server && npm test`` — **15 tests**: `converter.test.js`` (XSS, PDF ``%PDF-``, WebP ``RIFF``, CSV), `fileValidator`` (PNG/JSON), `auth.integration`` (register+2-step, bad pwd), `conversion`` (4 routes, bad format), `security`` (oversize 413, binary 400, XSS 200 + `<script>`; escaped).
- ``npm run lint`` — **0 errors** (eslint ignores ``_next``).
- ``npm audit`` — **0 vulns** (qs ``6.16.0`` override).
- ``curl /api/health`` `!`` ``{"status":"ok"}``.

8. Frontend GUI — What It Is

Web GUI, not desktop: `client/index.html:135`` two cards (auth + converter), `client/app.js:22`` derives ``API_BASE`` from `meta/localStorage/5500`, XHR `upload.progress`` shows `Uploading... 42%`, `clearAuth`` on 401, `logoutBtn``, `innerText`` prevents XSS. `client/dev-serve.js:1`` fixes `serve -o`` removal — ``npm run dev`` spawns `serve`` + `xdg-open``.

9. Professor Q&A — Ready Answers

****Q: Why not in-memory Map?**** A: Lost on restart, no FK/UNIQUE/index, no audit, no `EXPLAIN`. Brief required `users | sessions | conversion\_jobs | audit\_logs`.

****Q: Why hash JWT?**** A: Zero-trust: stolen DB shouldn't give bearer token; we store SHA256, verify against incoming.

****Q: Why SQLite not Postgres?**** A: Single file, zero infra, WAL concurrency, perfect for portfolio + professor demo; migration code identical concept.

****Q: File deleted after convert?**** A: Yes, never written — `multer.memoryStorage` + `res.send(Buffer)`; only `conversion\_jobs` row remains.

****Q: Show transaction.**** A: `server/userDb.js:98` `BEGIN IMMEDIATE !` COMMIT/ROLLBACK` in register.

****Q: Show injection prevention.**** A: `'?` + `ALLOWED\_USER\_COLUMNS` whitelist + `validateEmail` regex.

10. Checklist For Submission (Do Before Sending)

- [] ``cp server/.env.example server/.env.local`` with real ``JWT_SECRET``
- [] ``npm test --prefix server`` 15 passing + ``npm run lint`` clean
- [] ``npm run seed --prefix server`` !' 6 users visible
- [] ``xdg-open docs/schema.html`` !' light printable !' Ctrl+P Save PDF
- [] Screenshot ``assets/`` already in README
- [] ``git log --oneline main..local_commit`` shows humanized commits (no push yet)
- [] Attach ``docs/Report_SecureConvert_Armando_Ruggeri.pdf`` + this guide + ``docs/db-schema.md``

****Prepared for Prof. Armando Ruggeri — Good luck! Print this 4 pages.****